



Éléments non testés

Historique des versions

Version	Par qui ?	Quand ?	Quoi ?	Pourquoi ?
1.0	Valentin Fiche	20/03/2025	Tout	Création du document



Sommaire

Introduction	4
1) Component flame	5
2) Component tilemap et tilesheet	5
3) Api	5
4) Theme	6
5) main	6
6) Component play	6
7) Callbacks	7
8) Object component	7
9) Editor	7

Introduction

Dans le cadre de notre projet, certains fichiers ne peuvent pas être testés de manière automatisée en raison de leur nature spécifique. Il peut s'agir de composants fortement liés à l'interface utilisateur, de fichiers dépendants d'éléments externes non simulables, ou encore de modules dont le comportement ne peut être validé par des tests unitaires classiques.

Dans le cas des composants de jeux, cela concerne notamment les éléments liés à la physique, aux animations, aux interactions en temps réel ou aux effets visuels et sonores, qui nécessitent une exécution dans un environnement graphique pour être correctement évalués.

Ce fichier a pour objectif de recenser ces éléments afin de garantir une meilleure visibilité sur les parties du projet qui échappent aux tests automatisés. Il permet également d'identifier les composants nécessitant une validation manuelle ou une approche alternative pour assurer leur bon fonctionnement.

1) Component flame

Flame **n'est pas facilement testable avec des tests unitaires** en raison de sa dépendance au **rendu graphique et physique**, à la **boucle de jeu** en temps réel et aux **interactions utilisateur**. Les tests unitaires ne peuvent pas vérifier visuellement l'affichage ou simuler les entrées utilisateur.

chemin : lib/models/component_flame

2) Component tilemap et tileset

Les **Tilemaps** et **Tilesets** dans Flame ne sont pas facilement testables avec des tests unitaires en raison de leur dépendance au rendu graphique et à la gestion des ressources. Tester ces éléments implique de vérifier l'affichage correct des tuiles et de leur disposition, ce qui nécessite un environnement visuel qui ne peut pas être simulé facilement dans un test unitaire classique

chemin : lib/models/component_fields/tilemap et
lib/models/component_fields/tileset

3) Api

Il est possible de tester l'API en local en fournissant les clés API nécessaires pour effectuer les requêtes et valider les réponses. Cependant, lors de l'intégration continue (CI/CD), il devient impossible de tester directement l'API de la même manière, car les clés API ne sont pas fournies dans le dépôt GitHub pour des raisons de sécurité

chemin : lib/services/api.dart

4) Theme

Il est difficile de tester ce fichier car il définit uniquement une configuration statique du thème de l'application, sans logique ou comportement dynamique à tester. De plus, le thème est directement lié au rendu des widgets Flutter, ce qui nécessite un test d'intégration plutôt qu'un test unitaire.

chemin : lib/theme.dart

5) main

Il est difficile de tester ce fichier de manière unitaire car il dépend de l'environnement d'exécution (comme le fichier .env), de l'initialisation de composants UI complexes (comme MaterialApp et TabController), et d'interactions utilisateur qui nécessitent un cycle de vie complet de l'application Flutter. Les tests unitaires sont mieux adaptés aux logiques et données isolées, pas aux interactions UI

chemin : lib/main.dart

6) Component play

Les fichiers qui dépendent du moteur de jeu Flame, comme celui-ci, ne sont pas facilement testables car ils manipulent des objets visuels et des interactions physiques en temps réel. Ils dépendent de la scène graphique, de la caméra, et du rendu à l'écran, ce qui les rend difficiles à tester sans une interface graphique. De plus, la logique de jeu implique des mises à jour continues et des événements liés à l'animation et à l'interaction de l'utilisateur, ce qui complique leur test en isolation sans une exécution réelle du jeu.

chemin : lib/pages/play

7) Callbacks

Les fichiers Callbacks sont de simple classe de conteneur de fonctions (callbacks) sans logique propre. Il ne nécessite pas de tests unitaires car il ne contient ni traitement, ni état, ni conditions. Sans mocks, on peut seulement tester que l'instance se crée sans erreur, mais pas le comportement réel des fonctions.

chemin :

lib/pages/my_games/game_editor/editor/bottom_bar_callbacks.dart et
lib/pages/my_games/game_editor/editor/editor_callbacks.dart

8) Object component

On ne peut pas tester unitairement ce fichier car il dépend directement de forge2d, une librairie orientée physique et graphique difficile à simuler en dehors d'un environnement Flutter/Flame.

chemin :

lib/pages/my_games/game_editor/editor/object_scene_component.dart
lib/pages/my_games/game_editor/editor/object_component.dart
lib/pages/my_games/game_editor/editor/object_ui_component.dart

9) Editor

On ne peut pas tester ce fichier directement car il dépend de bibliothèques spécifiques comme Forge2D, Flame et Blocky, qui nécessitent un environnement d'exécution adapté.

chemin : lib/pages/my_games/game_editor/editor/editor.dart et
lib/pages/my_games/game_editor/editor/editor_page.dart

10) Bottom bar

Ce fichier ne peut pas être testé directement car il dépend du moteur Flame, charge des ressources graphiques de manière asynchrone, et nécessite un contexte de jeu pour fonctionner correctement.

chemin :

lib/pages/my_games/game_editor/editor/bottom_bar_button_canvas_component.dart et

lib/pages/my_games/game_editor/editor/bottom_bar_button_component.dart et

lib/pages/my_games/game_editor/editor/bottom_bar_button_mode_component.dart et

lib/pages/my_games/game_editor/editor/bottom_bar_callbacks.dart et

lib/pages/my_games/game_editor/editor/bottom_bar_component.dart

et lib/pages/my_games/game_editor/editor/side_bar_component.dart

